

Onglet 1

Compte rendu de la SAE-CYBER 01



Zakaria KESRAOUI
Ruhollah Sherzad
Ethel Sanquer

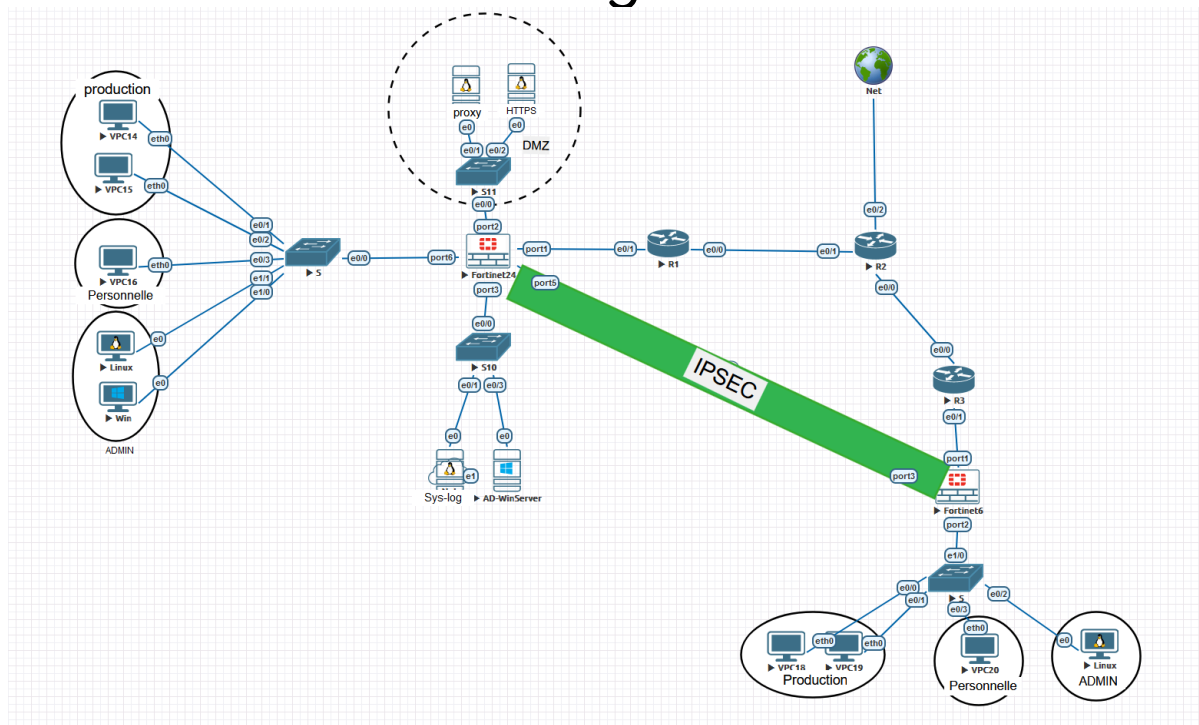
INTRODUCTION

On est partis sur une infrastructure basée sur les règles de l'ANSSI pour sécuriser un réseau multisite. On a configuré des pare-feu FortiGate et des tunnels IPsec pour isoler les flux, tout en centralisant les logs sur un serveur Ubuntu avec Wazuh pour la surveillance. Le but était de passer d'un simple labo à un environnement pro où chaque accès est contrôlé et chaque action est tracée. Ce rapport explique comment on a mis en place ces mesures concrètement pour protéger le réseau.

Sommaire

I. Architecture et Segmentation.....	4
II. Accès WAN et Connectivité.....	5
III. Sécurité Périmétrique (Pare-feu).....	9
IV. Services Réseau (Windows Server & DNSSEC).....	11
V. Proxy et Flux HTTPS.....	14
“Full Name:.....	18
URI:ldap:///CN=domain-WIN-8V4S3MFT74T-CA,CN=WIN-8V4S3MFT74T...” On voit le nom de Windows Server.....	18
Pourquoi ce transfert est-il nécessaire ?.....	18
VI. Détection et Prévention (IDS/IPS).....	19
VIII. Supervision et SIEM (Wazuh).....	21
Conclusion.....	24

I. Architecture et Segmentation



Nous avons choisi cette architecture parce qu'elle répond directement au besoin de protéger les données critiques tout en gardant une infrastructure gérable. En plaçant un pare-feu FortiGate au centre, nous avons voulu créer un point de contrôle unique capable de filtrer chaque échange entre l'interne, l'externe et les serveurs. Cette segmentation en zones a été choisie pour éviter qu'une compromission sur un poste utilisateur ou un serveur en DMZ ne permette à un attaquant de rebondir sur tout le réseau.

Un tunnel IPsec a été intégré pour répondre à la problématique du travail multisite, garantissant que nos flux d'administration et nos données métiers restent confidentiels lorsqu'ils passent par Internet. Enfin, nous avons ajouté une brique de supervision avec Wazuh car on a estimé qu'une sécurité efficace ne s'arrête pas au blocage, mais nécessite une visibilité constante. Cette architecture nous permet donc de respecter les exigences de l'ANSSI en termes de cloisonnement, de chiffrement et de traçabilité, tout en restant sur une structure simple et performante.

II. Accès WAN et Connectivité

Nous avons configuré l'accès WAN en commençant par les routeurs de bordure pour qu'ils gèrent le NAT. L'idée était de masquer nos adresses IP privées derrière une adresse publique unique, ce qui permet à tout notre réseau interne de sortir sur Internet tout en restant invisible depuis l'extérieur.

Les commandes proposées :

R1 :

```
configure terminal
! Interface "Publique" vers R2
interface ethernet 0/0
 ip address 172.16.12.2 255.255.255.252
 no shutdown
exit

! Interface "Privée" vers Fortinet5
interface ethernet 0/1
 ip address 10.62.120.1 255.255.255.252
 no shutdown
exit
```

Route par défaut pour envoyer le trafic vers R2 :

```
ip route 0.0.0.0 0.0.0.0 172.16.12.1
```

R2 :

```
configure terminal
interface ethernet 0/2
 no ip address dhcp
 ip address 172.16.254.2 255.255.255.0
 no shutdown
exit
```

On supprime l'ancienne route par défaut s'il y en avait une :

```
no ip route 0.0.0.0 0.0.0.0
```

On crée la nouvelle route vers le Linux EVE-NG :

```
ip route 0.0.0.0 0.0.0.0 172.16.254.1
```

```
! Lien vers R1
interface ethernet 0/1
 ip address 172.16.12.1 255.255.255.252
 ip nat inside
 no shutdown
exit
```

R3:

```
configure terminal
! Interface "Publique" vers R2
interface ethernet 0/0
 ip address 172.16.23.2 255.255.255.252
 no shutdown
 exit
```

```
! Interface "Privée" vers Fortinet6
interface ethernet 0/1
 ip address 10.62.121.1 255.255.255.252
 no shutdown
 exit
```

Route par défaut pour envoyer le trafic vers R2 :

```
ip route 0.0.0.0 0.0.0.0 172.16.23.1
```

! Lien vers R3

```
interface ethernet 0/0
 ip address 172.16.23.1 255.255.255.252
 ip nat inside
 no shutdown
```

On s'assure que l'interface vers Linux/Internet est OUTSIDE :

```
interface ethernet 0/2
 ip nat outside
 exit
```

On s'assure que les interfaces vers R1 et R3 sont INSIDE :

```
interface ethernet 0/1
 ip nat inside
 exit
interface ethernet 0/0
 ip nat inside
 exit
```

On autorise absolument tout le trafic interne à être traduit (ACL) :

```
access-list 1 permit any
```

On active la traduction magique (PAT/Overload) :

```
ip nat inside source list 1 interface ethernet 0/2 overload
```

Au niveau des switches, le trafic a été segmenté en créant des VLANs pour isoler les différents services. Nous avons mis en place des liaisons "Trunk" sur les ports d'interconnexion pour que les flux de tous nos réseaux puissent circuler proprement entre les switches et remonter jusqu'au pare-feu sans se mélanger.

Les commandes proposées :

S1-S2 :

```
enable
configure terminal
```

Création des VLANs :

```
vlan 10
  name Admin
vlan 20
  name Personnel
vlan 30
  name Production
exit
```

Configuration du lien Trunk vers le FortiGate (e1/0) :

```
interface ethernet 1/0
  switchport trunk encapsulation dot1q
  switchport mode trunk
exit
```

Assignment du PC Linux au VLAN Admin :

```
interface ethernet 0/3
  switchport mode access
  switchport access vlan 10
  spanning-tree portfast
exit
```

Assignment des VPC Personnel :

```
interface range ethernet 0/0 - 1
  switchport mode access
  switchport access vlan 20
  spanning-tree portfast
exit
```

Assignment du VPC Production :

```
interface ethernet 0/2
  switchport mode access
  switchport access vlan 30
  spanning-tree portfast
exit
```


Nous avons segmenté l'architecture en créant trois VLANs distincts sur le port 2 pour isoler les flux d'administration, personnels et de production, tout en activant des serveurs DHCP sur chaque interface pour automatiser l'adressage des postes. Le port 1 a été configuré comme interface WAN avec une IP fixe et les accès d'administration

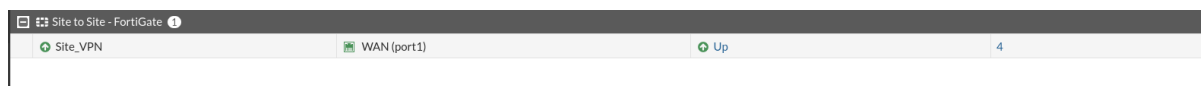
Physical Interface 3							
port2	Physical Interface		0.0.0.0/0.0.0.0				3
• VLAN10_ADMIN	VLAN		10.62.101.1/255.255.255.0	PING	2	10.62.101.2-10.62.101.254	7
• VLAN20_Perso	VLAN		10.62.102.1/255.255.255.0	PING	2	10.62.102.2-10.62.102.254	4
• VLAN30_PRODU	VLAN		10.62.103.1/255.255.255.0	PING	2	10.62.103.2-10.62.103.254	4
port3	Physical Interface		172.16.254.254/255.255.255.0	PING HTTPS SSH HTTP			0
port4	Physical Interface		0.0.0.0/0.0.0.0				0
WAN (port1)	Physical Interface		10.62.121.2/255.255.255.252	PING HTTPS SSH HTTP FMCG-Access			6

Des politiques de sécurité spécifiques ont été créées pour autoriser chaque VLAN à sortir vers l'interface WAN.

+	VLAN10_ADMIN → WAN (port1)	1
+	VLAN20_Perso → WAN (port1)	1
+	VLAN30_PRODU → WAN (port1)	1
+	Implicit	1

III. Sécurité Périmétrique (Pare-feu)

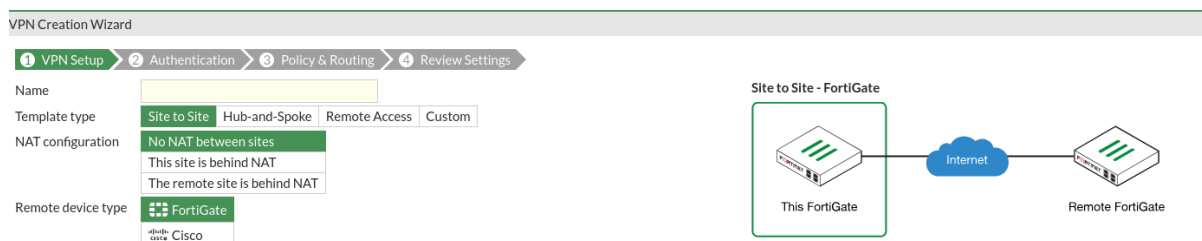
Toute notre logique de sécurité périmétrique a été concentrée sur le FortiGate, en commençant par le montage d'un tunnel VPN IPsec entre les deux sites. L'objectif était de créer un lien chiffré et transparent pour que nos flux d'administration et nos données internes circulent en toute sécurité sur le réseau public sans risquer d'être interceptés.



Explication:

L'IPsec fonctionne comme un tunnel blindé qui protège les données quand elles traversent un réseau non sûr, comme Internet. Pour que ça marche, on utilise deux protocoles principaux : l'**AH** qui garantit que personne n'a modifié les données en route, et l'**ESP** qui chiffre le contenu pour que personne ne puisse le lire. Tout commence par une phase de négociation (appelée **IKE**) où les deux équipements s'entendent sur les clés de chiffrement et les algorithmes à utiliser avant d'ouvrir le tunnel pour faire passer les paquets.

Sur le FortiGate, cela a été configuré en mode "Interface Tunnel". Le VPN apparaît donc comme une interface réseau classique sur laquelle on peut appliquer des règles de sécurité. On a d'abord défini l'adresse IP publique de la passerelle distante et les clés pré-partagées (PSK) pour l'authentification. Ensuite, on a précisé les sous-réseaux locaux et distants qui ont le droit de communiquer à travers ce lien sécurisé.



Une fois le tunnel monté, on a dû créer des politiques de pare-feu spécifiques pour autoriser le trafic à entrer et sortir par cette interface VPN. L'ajout des routes statiques a ensuite indiqué au FortiGate que tout le trafic destiné au site distant doit être envoyé vers le tunnel IPsec plutôt que vers la passerelle Internet classique. C'est ce qui permet aux deux sites de se voir de manière transparente et chiffrée.

Le ping entre les deux sites:

```
Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::b1a6:d8a3:1518:4905%9
    IPv4 Address. . . . . : 10.62.111.3
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 10.62.111.1

C:\Users\zkesraoui>ping 10.62.101.2

Pinging 10.62.101.2 with 32 bytes of data:
Reply from 10.62.101.2: bytes=32 time=5ms TTL=62
Reply from 10.62.101.2: bytes=32 time=3ms TTL=62
Reply from 10.62.101.2: bytes=32 time=4ms TTL=62
Reply from 10.62.101.2: bytes=32 time=3ms TTL=62

Ping statistics for 10.62.101.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 3ms, Maximum = 5ms, Average = 3ms

C:\Users\zkesraoui>
```

Ensuite nous avons structuré le routage inter-VLAN pour que le VLAN Admin soit le seul à avoir une visibilité complète sur l'ensemble de l'infrastructure. On a configuré des règles de flux permettant aux administrateurs d'initier des connexions vers les serveurs, le LAN et la DMZ pour la maintenance, tout en interdisant strictement l'inverse.

Parefeu du site 2:

Site_VPN → VLAN10_ADMIN
VLAN10_ADMIN → Site_VPN
VLAN10_ADMIN → VLAN20_Perso
VLAN10_ADMIN → VLAN30_PRODU
VLAN10_ADMIN → WAN (port1)
VLAN20_Perso → WAN (port1)
VLAN30_PRODU → WAN (port1)

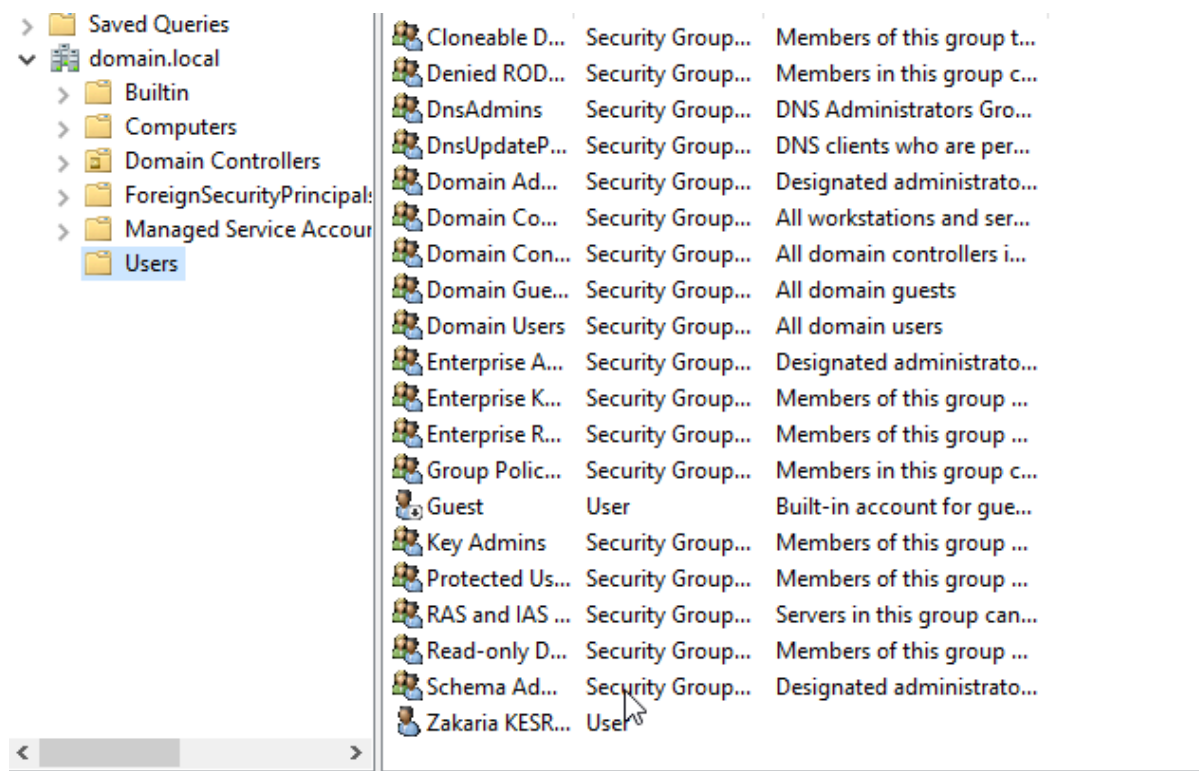
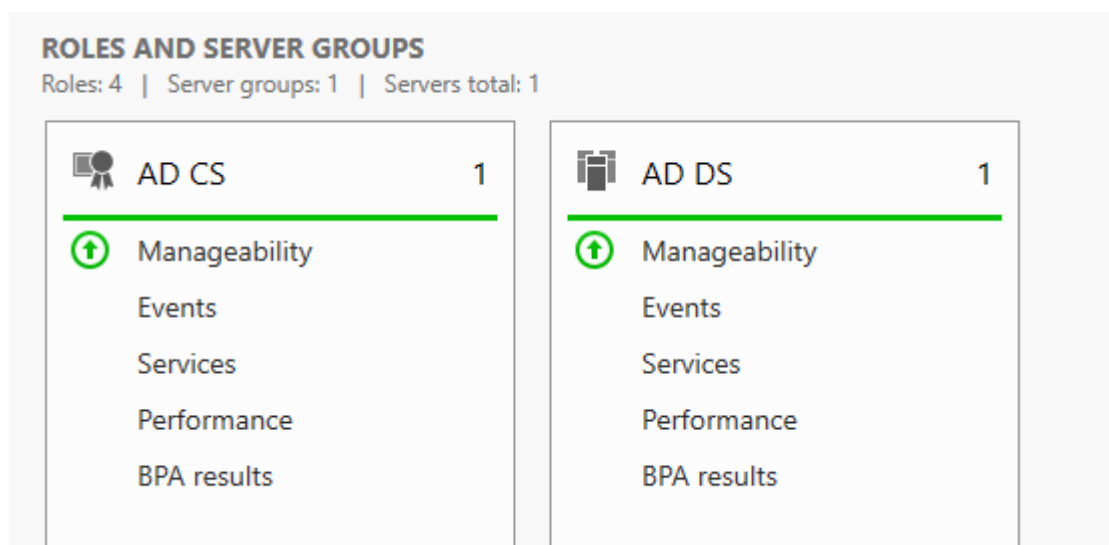
Pour les autres réseaux comme le VLAN Perso ou la DMZ, on a appliqué un cloisonnement étanche où chaque zone reste isolée de sa voisine. Si un utilisateur du LAN tente de joindre un équipement d'administration ou un serveur sensible, le trafic est immédiatement rejeté par le pare-feu car aucune règle ne l'autorise, limitant ainsi les risques de mouvements latéraux en cas de compromission d'un poste.

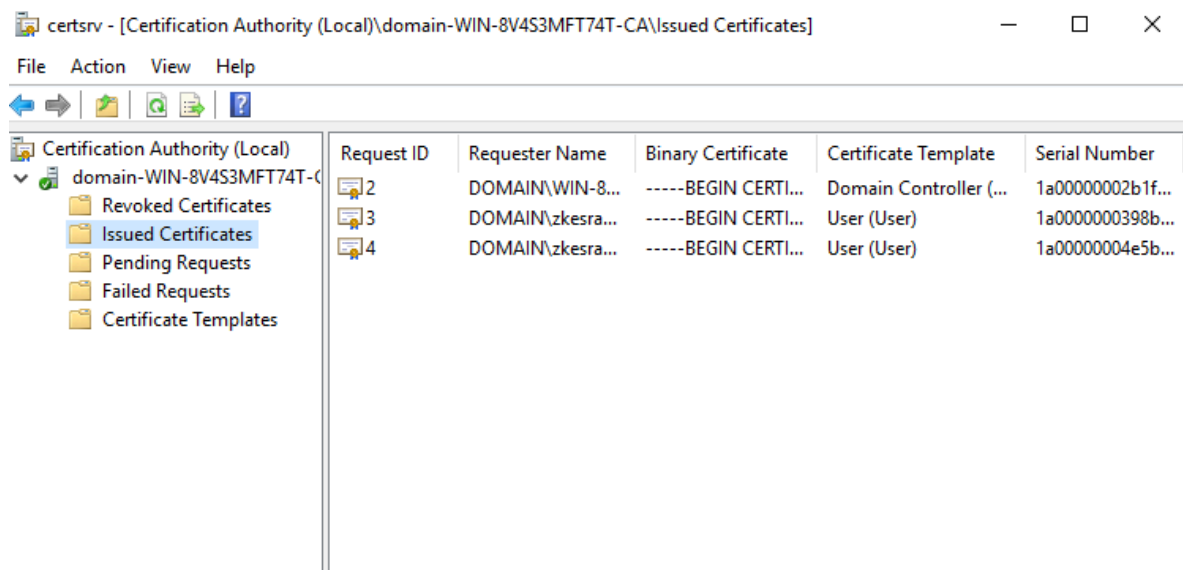
Parefeu du site 1:

SERVEUR_DMZ (port2) → SERVEUR_INTERNE (port3)
SERVEUR_DMZ (port2) → VLAN_ADMIN
SERVEUR_DMZ (port2) → WAN (port1)
SERVEUR_INTERNE (port3) → port5
SERVEUR_INTERNE (port3) → SERVEUR_DMZ (port2)
Site_VPN → VLAN_ADMIN
VLAN_ADMIN → SERVEUR_DMZ (port2)
VLAN_ADMIN → SERVEUR_INTERNE (port3)
VLAN_ADMIN → Site_VPN
VLAN_ADMIN → WAN (port1)

IV. Services Réseau (Windows Server & DNSSEC)

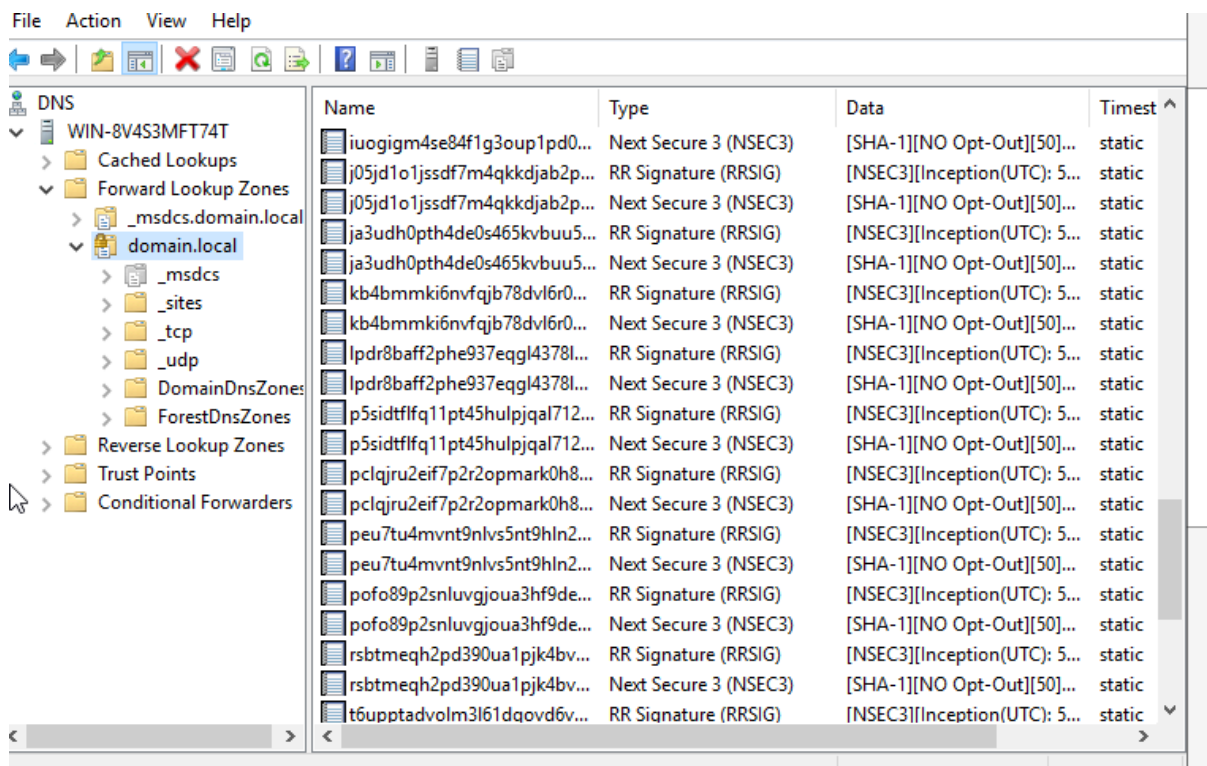
Nous avons déployé un serveur Windows pour centraliser la gestion des utilisateurs et des machines à travers un domaine Active Directory. Pour renforcer la sécurité des échanges internes, on a ajouté le rôle AD CS afin de mettre en place notre propre autorité de certification. Cela nous permet de délivrer des certificats numériques aux équipements et aux services, garantissant ainsi l'identité des serveurs et le chiffrement des communications au sein de notre infrastructure.



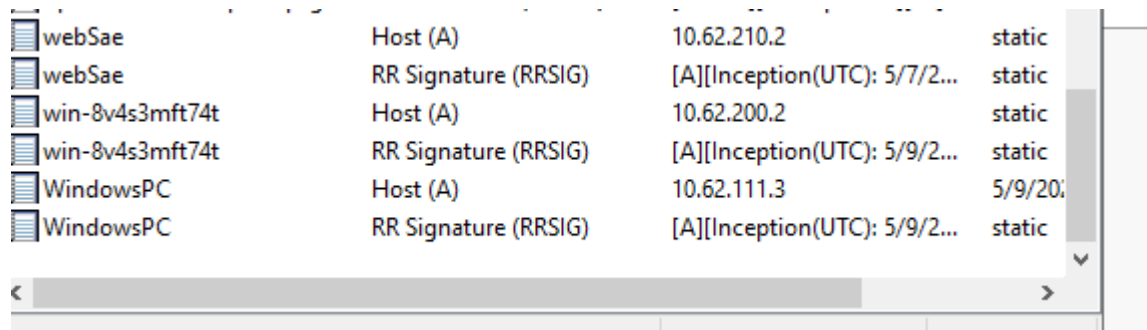


Les deux lignes "User" pour l'utilisateur zkesra prouvent que tes comptes clients ont bien reçu des certificats personnels.

Par-dessus cette base, on a activé DNSSEC sur notre serveur DNS pour protéger le mécanisme de résolution de noms, qui est souvent une cible privilégiée. En signant numériquement les zones DNS, on s'assure qu'un attaquant ne peut pas détourner le trafic en envoyant de fausses réponses (empoisonnement de cache). C'est une protection indispensable pour garantir que, lorsqu'un utilisateur cherche à joindre un service interne, il soit dirigé vers la bonne adresse IP sans risque d'interception.



La présence de l'enregistrement **RRSIG** (Resource Record Signature) dans la console DNS : sa présence confirme que la zone est protégée contre l'empoisonnement de cache et que chaque réponse DNS est authentifiée par une signature numérique.



webSae	Host (A)	10.62.210.2	static
webSae	RR Signature (RRSIG)	[A][Inception(UTC): 5/7/2...	static
win-8v4s3mft74t	Host (A)	10.62.200.2	static
win-8v4s3mft74t	RR Signature (RRSIG)	[A][Inception(UTC): 5/9/2...	static
WindowsPC	Host (A)	10.62.111.3	5/9/20...
WindowsPC	RR Signature (RRSIG)	[A][Inception(UTC): 5/9/2...	static

On a choisi d'activer **DNSSEC directement sur l'Active Directory** car dans notre architecture, c'est lui qui détient l'autorité sur toute la zone de domaine interne. Comme l'AD gère déjà l'authentification et les services critiques via les enregistrements SRV, sécuriser le DNS à la source permet de protéger l'annuaire contre l'usurpation d'identité ou le détournement de session sans avoir à gérer une machine supplémentaire

Faire du DNSSEC sur Linux avec Bind ou Unbound est très performant pour des résolveurs publics, mais dans notre cas, cela aurait complexifié la réplication des données. En restant sur Windows, la signature des zones est **intégrée nativement à la réplication Active Directory** : dès qu'on signe une zone sur un contrôleur de domaine, elle est automatiquement propagée et sécurisée sur tous les autres, ce qui garantit une cohérence parfaite de la sécurité sur tout notre réseau privé.

V. Proxy et Flux HTTPS

Nous avons mis en place un proxy pour contrôler tout ce qui sort du réseau vers Internet et s'assurer que les utilisateurs ne visitent pas de sites dangereux ou non autorisés. Au lieu de laisser chaque machine sortir librement, le proxy agit comme un filtre intermédiaire qui vérifie la catégorie du site et applique nos politiques de filtrage web, ce qui réduit considérablement les risques d'infection par des sites malveillants.

On a installé le paquet `squid` sur notre serveur Ubuntu pour qu'il serve de passerelle unique vers le Web. Son rôle est de filtrer les URL demandées par les clients du LAN et de mettre en cache les pages pour économiser de la bande passante.

La configuration du fichier `squid.conf`

Le fichier principal `/etc/squid/squid.conf` a dû être modifié pour définir qui a le droit de naviguer.

- Définition des ACL : On a créé des "Access Control Lists" pour identifier notre réseau local (ex: `acl lan_reseau src 10.62.103.0/24`).
- Gestion des accès : On a ajouté des règles `http_access allow lan_reseau` pour autoriser notre LAN et `http_access deny all` pour bloquer tout le reste par sécurité.
- Le port : Par défaut, on a gardé le port 3128.

Le filtrage par domaine :

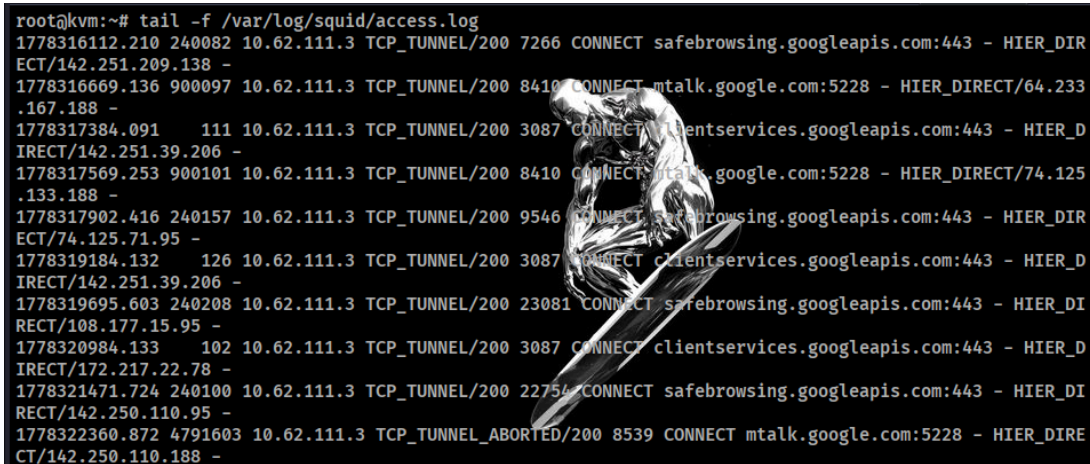
Pour bloquer des sites précis, on a créé un fichier simple (ex: `/etc/squid/sites_bloques.txt`) contenant les domaines interdits. Dans la conf Squid, on a ajouté une règle qui lit ce fichier et bloque l'accès à tout ce qui s'y trouve.

Les commandes clés qu'on a utilisées :

- `sudo systemctl restart squid` : Pour appliquer nos changements de config.
- `sudo squid -k reconfigure` : Pour recharger la liste des sites bloqués sans couper le service.
- `tail -f /var/log/squid/access.log`

Le défi avec le Web aujourd'hui, c'est que presque tout le trafic est chiffré en HTTPS, ce qui rend le filtrage classique "aveugle". Pour résoudre ça, on a activé l'inspection SSL (Deep Inspection) sur le pare-feu. On utilise le certificat de notre autorité de certification (AD CS) pour que le FortiGate puisse ouvrir les paquets chiffrés, les analyser à la recherche de virus ou de scripts malveillants, puis les rechiffrer avant de les envoyer à l'utilisateur.

Résultat:



```
root@kvm:~# tail -f /var/log/squid/access.log
1778316112.210 240082 10.62.111.3 TCP_TUNNEL/200 7266 CONNECT safebrowsing.googleapis.com:443 - HIER_DIRECT/142.251.209.138 -
1778316669.136 900097 10.62.111.3 TCP_TUNNEL/200 8410 CONNECT mtalk.google.com:5228 - HIER_DIRECT/64.233.167.188 -
1778317384.091 111 10.62.111.3 TCP_TUNNEL/200 3087 CONNECT clientservices.googleapis.com:443 - HIER_DIRECT/142.251.39.206 -
1778317569.253 900101 10.62.111.3 TCP_TUNNEL/200 8410 CONNECT mtalk.google.com:5228 - HIER_DIRECT/74.125.133.188 -
1778317902.416 240157 10.62.111.3 TCP_TUNNEL/200 9546 CONNECT safebrowsing.googleapis.com:443 - HIER_DIRECT/74.125.71.95 -
1778319184.132 126 10.62.111.3 TCP_TUNNEL/200 3087 CONNECT clientservices.googleapis.com:443 - HIER_DIRECT/142.251.39.206 -
1778319695.603 240208 10.62.111.3 TCP_TUNNEL/200 23081 CONNECT safebrowsing.googleapis.com:443 - HIER_DIRECT/108.177.15.95 -
1778320984.133 102 10.62.111.3 TCP_TUNNEL/200 3087 CONNECT clientservices.googleapis.com:443 - HIER_DIRECT/172.217.22.78 -
1778321471.724 240100 10.62.111.3 TCP_TUNNEL/200 22754 CONNECT safebrowsing.googleapis.com:443 - HIER_DIRECT/142.250.110.95 -
1778322360.872 4791603 10.62.111.3 TCP_TUNNEL_ABORTED/200 8539 CONNECT mtalk.google.com:5228 - HIER_DIRECT/142.250.110.188 -
```

Le fonctionnement du proxy Squid a été validé en analysant les fichiers de logs en temps réel. La capture montre que le client 10.62.111.3 transmet ses requêtes HTTPS via la méthode CONNECT sur le port 3128. Le statut TCP_TUNNEL/200 confirme que le proxy intercepte, traite et autorise correctement les flux sortants du LAN vers Internet

HTTPS:

L'objectif de cette étape est d'installer un service Web robuste sur la machine Linux et de le configurer pour qu'il communique de manière chiffrée en utilisant un certificat approuvé par l'infrastructure Windows de l'entreprise.

1. Installation et préparation d'Apache

La première phase consiste à installer le paquet serveur et à activer les outils nécessaires à la gestion du protocole SSL/TLS.

- Installation du service : `apt update & apt install apache2 -y`
- Activation du module SSL : `a2enmod ssl`. Cette commande permet à Apache de charger les bibliothèques nécessaires au chiffrement des flux.

Pour intégrer notre serveur Apache à la PKI de l'entreprise, nous avons utilisé l'outil OpenSSL pour générer un couple de clés cryptographiques. Ce processus est le fondement de la sécurité SSL/TLS :

1. Génération de la clé privée et de la clé publique : Nous avons créé une clé privée (le fichier **.key**), qui est restée confidentielle sur le serveur Linux. À partir de cette clé, OpenSSL a extrait une clé publique encapsulée dans un fichier appelé CSR (Certificate Signing Request). Ce CSR est une demande officielle d'identité contenant la clé publique du serveur et ses informations (nom de domaine, organisation, etc.).

```
openssl req -nodes -newkey rsa:2048 -keyout websae.key -out websae.csr
```

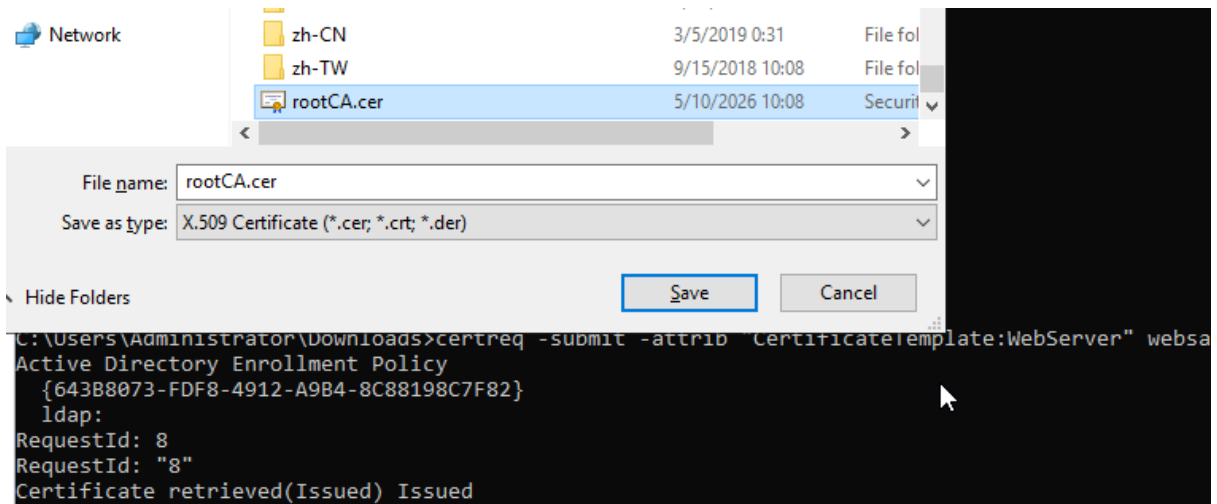
```
root@kvm:/etc/apache2/ss  
websae.csr websae.key  
root@kvm:/etc/apache2/
```

```
root@kvm:/etc/apache2/ssl# cat websae.csr
-----BEGIN CERTIFICATE REQUEST-----
MIIC5zCCAc8CAQAweDELMAkGA1UEBhMCZnIxZCZAJBgNVBAGMAmZyMQswCQYDVQQH
DAJmcjELMAkGA1UECgwCZnIxZCZAJBgNVBAsMAmZyMRwwGgYDVQQDBN3ZnJzYWUu
ZG9tYWluLmxvY2FsMRcwFQYJKoZIhvcNAQkBFgh6YWNremFjazCCASIhDQYJKoZI
hvcNAQEBBQADggEPADCCAQoCggEBANgx02KWKVHzilchdgOf6exhch0gf7AQxbtA
cF0zh78TwimiceSIb+BV+d8XEHSQLeCu8STsZF6R0PELGrbaSZVmVzBul9C0bLI
PILUL8XCCZjTQZBlFvJJTsRFRkXPgxn/HwZhS09qTJd17cYygLKgeHJ34ALzldHx
gdhDT5GPX4DmAgE66SwP6uTapymSYhDuG9l7UuMRlEydpd1J4sPdqrI1Av7k9DF
/eBk5Bisu8Ja5FIi0qnFEQqSTxhBXtcySHjVCwEAZEGBFUMVnyXETHKAb7aPUera
hUeVPS70EU+5DSnSpF7h5YpXUBE+gJ+sepaGdLRtQ4jQ30owY40CAwEAAaAqMBMG
CSqGSIb3DQEJAJEGDARYb290MBMGCSqGSIb3DQEJBzEGDARYb290MA0GCSqGSIb3
DQEBcwUAA4IBAQBnqacRcKwbevwpJUnWeKQ05hR53n30pA3vij+VMBvT8We0sx1q
Y+h+pe9h6pEXvjRGVa9PCeESWTy00S7rvj6UDrfcRIOx9HPdwB7vQf8A/WQlgMZN
ZxoLV3/3L2n2LmaRvAWmtqPLQYcKFLba1BAid0Hzv/B7UX0rT0CGbpbH0dBzSI3G
v4RXfauWp/S6uRcEB7rvaRr1FXg7oBcFMm6IfbSbUqlGvWEUg2n1FDYQ0dwhd0/
UqivETppjcpzop9o/VyTrZcdu0Yd+R1lodRtHWtCBagZJ7jjNgv9fLGMe+4HGj5C
XM1K+CytDRwoSjaSREx2j3w2eHhkAYx1FOq+
-----END CERTIFICATE REQUEST-----
root@kvm:/etc/apache2/ssl#
```

2. Transfert et Signature par l'AD : Nous avons ensuite transféré ce fichier CSR (contenant la clé publique) vers le serveur Active Directory (AD CS). Dans cette architecture, l'AD fait office d'autorité de certification. Son rôle est de vérifier notre demande et de la "signer" numériquement avec sa propre clé privée.

```
certreq -submit -attrib "CertificateTemplate:WebServer"  
C:\Users\Administrateur\Desktop\websae.csr
```

```
C:\Users\Administrator\Downloads>certreq -submit -attrib "CertificateTemplate:WebServer" websae.csr
Active Directory Enrollment Policy
{643B8073-FDF8-4912-A9B4-8C88198C7F82}
ldap:
RequestId: 7
RequestId: "7"
Certificate retrieved(Issued) Issued
```



Une fenêtre va s'ouvrir pour demander de sélectionner l'Autorité de Certification (CA). Validez.

Windows va demander où enregistrer le certificat signé

3. Le certificat final : Une fois signé par l'AD, le fichier nous est revenu sous forme de certificat (le fichier **.crt**). Ce certificat contient désormais notre clé publique accompagnée de la signature numérique de l'AD.

On se transfère au serveur apache et on l'applique.

```
root@kvm:/etc/apache2/ssl# ls
websae.csr websae.key websae_signe.crt
root@kvm:/etc/apache2/ssl#
```

```
X509v3 CRL Distribution Points:

Full Name:
  URI:ldap:///CN=domain-WIN-8V4S3MFT74T-CA,CN=WIN-8V4S3MFT74T,CN=CDP,CN=Public%20Key%20S
ices,CN=Services,CN=Configuration,DC=domain,DC=local?certificateRevocationList?base?objectClass=cRLDi
tributionPoint

Authority Information Access:
  CA Issuers - URI:ldap:///CN=domain-WIN-8V4S3MFT74T-CA,CN=AIA,CN=Public%20Key%20Services,
Services,CN=Configuration,DC=domain,DC=local?cACertificate?base?objectClass=certificationAuthority

1.3.6.1.4.1.311.20.2;
```

"Full Name:

URI:ldap:///CN=domain-WIN-8V4S3MFT74T-CA,CN=WIN-8V4S3MFT74T..." On voit le nom de Windows Server.

Pourquoi ce transfert est-il nécessaire ?

Le transfert de la clé publique vers l'AD est indispensable pour que le serveur Windows puisse attester que notre serveur Linux est légitime. En signant notre clé publique, l'AD dit à tous les ordinateurs du réseau : *"Je garantis que cette clé publique appartient bien au serveur websae.domain.local"*. Sans cette signature, les clients ne pourraient pas vérifier l'authenticité du serveur et bloqueraient la connexion.

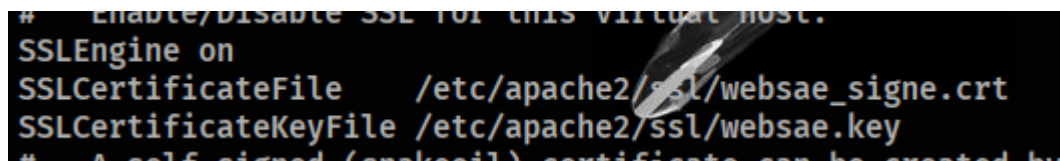
Configuration du VirtualHost SSL

La dernière étape consiste à modifier la configuration d'Apache pour qu'il utilise nos nouveaux fichiers au lieu des certificats auto-signés par défaut. Nous avons édité le fichier `/etc/apache2/sites-available/default-ssl.conf`

SSLEngine on

SSLCertificateFile /etc/apache2/ssl/websae_signe.crt

SSLCertificateKeyFile /etc/apache2/ssl/websae.key

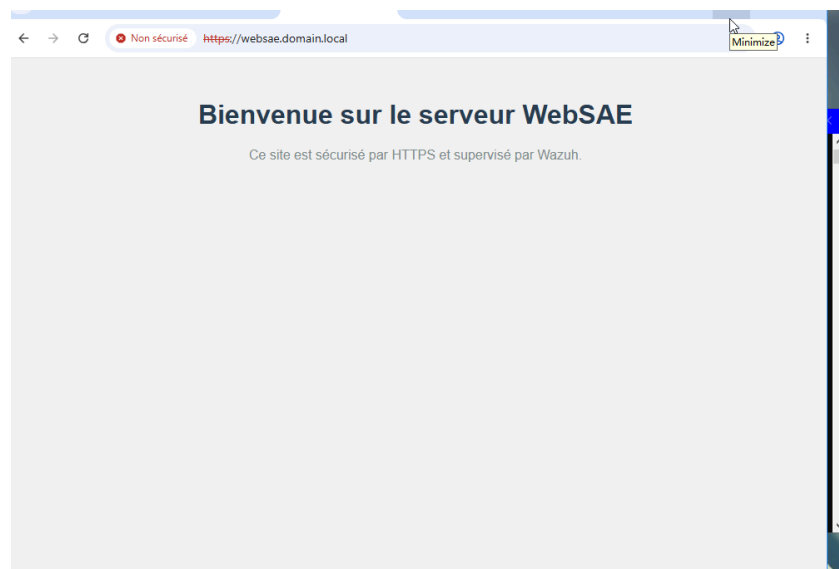


Finalisation et mise en service

Pour appliquer les paramètres, nous avons activé le site sécurisé et redémarré le service :

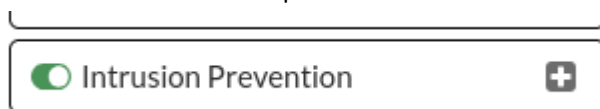
- `a2ensite default-ssl`
- `systemctl restart apache2`

Resultat:



VI. Détection et Prévention (IDS/IPS)

La mise en place d'un serveur Web et d'une autorité de certification assure la confidentialité, mais elle ne protège pas contre les tentatives d'attaques (exploits, scans de ports, force brute). Pour pallier cela, nous avons intégré une couche de sécurité proactive entre le WAN et le LAN.

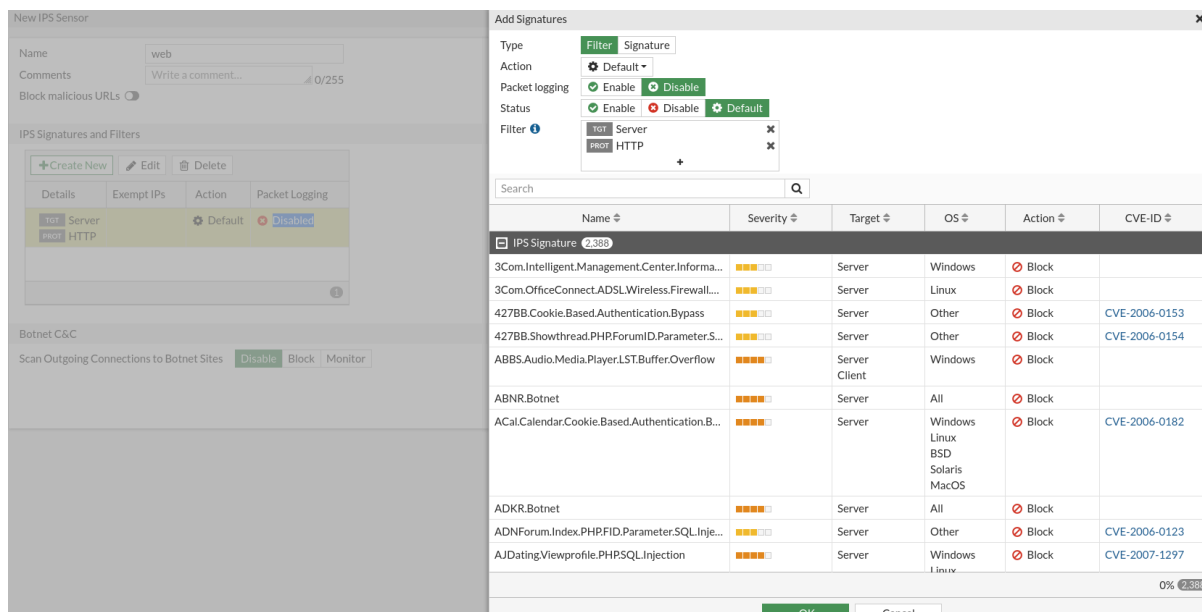


L'objectif est de surveiller en temps réel les paquets circulant sur le réseau afin d'identifier des signatures d'attaques connues ou des comportements suspects.

- **IDS (Détection)** : Analyse le trafic et génère une alerte en cas d'anomalie, sans interrompre la connexion.
- **IPS (Prévention)** : Va plus loin en bloquant automatiquement le trafic jugé malveillant (par exemple, en ajoutant une règle de pare-feu dynamique pour bannir une IP attaquante).

Nous avons utilisé des outils de référence (comme Snort ou Suricata) positionnés stratégiquement sur la passerelle ou le serveur pour intercepter le trafic.

- **Mise à jour des signatures** : Le système s'appuie sur une base de données de signatures (règles) mise à jour régulièrement pour reconnaître les menaces les plus récentes.

A screenshot of a "New IPS Sensor" configuration window. The left pane shows the "IPS Signatures and Filters" section with a table containing one entry: "Server" with action "Default" and status "Disabled". The right pane shows the "Add Signatures" dialog with a list of signatures. The "Filter" tab is active, showing a list of signatures with columns for Name, Severity, Target, OS, Action, and CVE-ID. The list includes various known attacks like "3Com.Intelligent.Management.Center.Information", "427BB.Cookie.Based.Authentication.Bypass", etc. The bottom of the window has "OK" and "Cancel" buttons.

Name	Severity	Target	OS	Action	CVE-ID
3Com.Intelligent.Management.Center.Information	High	Server	Windows	Block	
3Com.OfficeConnect.ADSL.Wireless.Firewall...	High	Server	Linux	Block	
427BB.Cookie.Based.Authentication.Bypass	High	Server	Other	Block	CVE-2006-0153
427BB.Showthread.PHPForumID.Parameter.S...	High	Server	Other	Block	CVE-2006-0154
ABBS.Audio.Media.Player.LST.Buffer.Overflow	High	Server Client	Windows	Block	
ABNR.Botnet	High	Server	All	Block	
ACal.Calendar.Cookie.Based.Authentication.B...	High	Server	Windows Linux BSD Solaris MacOS	Block	CVE-2006-0182
ADKR.Botnet	High	Server	All	Block	
ADNForum.Index.PHP.FID.Parameter.SQL.Inje...	High	Server	Other	Block	CVE-2006-0123
AJDating.Viewprofile.PHP.SQL.Injection	High	Server	Windows Linux	Block	CVE-2007-1297

Configuration des interfaces : L'outil est configuré en mode "Promiscuous" pour pouvoir lire l'intégralité du trafic passant par l'interface réseau, et non seulement celui qui lui est destiné.

Type	Filter	Signature
Action	Monitor ▾	
Packet logging	☒ Allow	Disabl
Status	Monitor	Disabl
Filter	Block	
	Reset	
	Default	+
	Quarantine	
Search		
Name		
IPS	☒	IPS web ▾
File Filter	☐	
SSL Inspection		SSL deep-inspection ▾
Decrypted Traffic Mirror	☐	

VIII. Supervision et SIEM (Wazuh)

La surveillance de notre infrastructure est centralisée par Wazuh, une solution de SIEM qui permet d'agréger les journaux d'événements de tous nos systèmes. Cette centralisation repose sur deux méthodes : l'installation d'agents légers sur les serveurs et l'utilisation du protocole Syslog pour les équipements réseau.

Configuration du Serveur Syslog Centralisé

Le serveur Syslog (basé sur rsyslog) a été configuré pour écouter les messages provenant du réseau. Nous avons dû modifier le fichier de configuration `/etc/rsyslog.conf` pour activer la réception UDP/TCP sur le port 514 :

```
# Activation de la réception UDP
module(load="imudp")
input(type="imudp" port="514")
# Activation de la réception TCP
module(load="imptcp")
input(type="imptcp" port="514")
```

Configuration des équipements réseau (Switch et Routeur)

Les équipements Cisco ne permettent pas l'installation d'agents. Nous avons donc configuré l'envoi des logs vers le serveur Wazuh via le réseau. Les commandes suivantes ont été saisies en mode configuration globale :

Sur le Switch et le Routeur :

```
conf t
logging on
logging host 10.62.200.3
logging trap informational
logging source-interface
exit
```

La commande `logging trap` définit le niveau de détail des logs envoyés, assurant ainsi que chaque connexion (Login/Logout) ou changement d'état d'interface soit enregistré par le SIEM.

Configuration du Pare-feu (Fortinet)

Pour le Fortigate, la configuration a été réalisée via la console (CLI) pour diriger les logs de trafic et de sécurité vers notre collecteur :

```
config log syslogd setting
    set status enable
    set server "10.62.200.3"
    set mode udp
    set port 514
```

end

Cette configuration permet à Wazuh d'analyser les tentatives d'accès bloquées par les règles de firewalling.

Configuration du Proxy (Squid sur Linux)

Le serveur Proxy génère des journaux critiques pour surveiller la navigation des utilisateurs. Nous avons configuré l'agent Wazuh sur le Linux pour qu'il lise en temps réel les fichiers de Squid :

Fichier de configuration : `/etc/squid/squid.conf`

à ajouter ou modifier:

```
# Envoi des logs d'accès vers syslog avec la catégorie 'local2'
access_log syslog:local2.info squid
```

Une fois que Squid transmet ses données à rsyslog sur la machine locale, ce dernier doit les rediriger vers l'IP du serveur Syslog central sur le réseau EVE-NG.

Fichier de configuration : `/etc/rsyslog.conf` (ou `/etc/rsyslog.d/99-squid.conf`)

Commande de transfert :

```
# Redirection de tous les messages local2 vers le collecteur
distant
```

```
local2.* @10.62.200.3:514
```

Et on redémarre le service squid

Configuration du Serveur Web (Apache HTTPS)

Par défaut, Linux utilise le service **rsyslog** pour gérer les messages système. Nous devons lui indiquer d'écouter les logs d'Apache et de les renvoyer sur le réseau.

- Fichier de configuration : `/etc/rsyslog.d/apache-to-syslog.conf` (ou directement dans `/etc/rsyslog.conf`).
- Commande pour l'envoi vers le serveur distant :

```
# Envoyer tous les messages locaux au serveur central via UDP (port 514)
```

```
*.* @10.62.200.3:514
```

Redémarrage de l'agent pour appliquer les changements et du service apache:
systemctl restart wozuh-agent
Pourquoi cette architecture ?

Le SIEM récupère ces flux et utilise des moteurs de règles pour corréler les données. Par exemple, si le Fortinet détecte un scan de port et que, simultanément, le serveur Apache enregistre des erreurs de connexion inhabituelles, Wazuh génère une alerte de priorité haute. Cette visibilité globale transforme des logs isolés en une véritable stratégie de défense proactive, permettant de réagir avant qu'une intrusion ne réussisse.

Résultat sur syslog:



```
info="name=port1,bytes=5746719/92414845,packets=47998/95000;" msg="Performance statistics: average CPU: 1, memory: 41, concurrent sessions: 35, setup-rate: 0"
May 11 06:58:54 10.62.200.1 date=2026-05-10 time=15:58:54 devname="FortiGate-VM64-KVN" devid="FGVMEVRM27
T94VEB" eventtime=1778653934100268390 tz="-0700" logid="0100026003" type="event" subtype="system" level=
"information" vd="root" logdesc="DHCP statistics" interface="VLAN ADMIN" total=253 used=0 msg="DHCP stat
istics"
May 11 06:58:54 10.62.200.1 date=2026-05-10 time=15:58:54 devname="FortiGate-VM64-KVN" devid="FGVMEVRM27
T94VEB" eventtime=1778653934100272778 tz="-0700" logid="0100026003" type="event" subtype="system" level=
"information" vd="root" logdesc="DHCP statistics" interface="VLAN PRODU" total=253 used=0 msg="DHCP stat
istics"
May 11 06:58:54 10.62.200.1 date=2026-05-10 time=15:58:55 devname="FortiGate-VM64-KVN" devid="FGVMEVRM27
T94VEB" eventtime=1778653934100273794 tz="-0700" logid="0100026003" type="event" subtype="system" level=
"information" vd="root" logdesc="DHCP statistics" interface="port2" total=253 used=1 msg="DHCP statisti
cs"
May 11 06:58:54 10.62.200.1 date=2026-05-10 time=15:58:55 devname="FortiGate-VM64-KVN" devid="FGVMEVRM27
T94VEB" eventtime=1778653934100274673 tz="-0700" logid="0100026003" type="event" subtype="system" level=
"information" vd="root" logdesc="DHCP statistics" interface="port3" total=253 used=1 msg="DHCP statisti
cs"
May 11 06:59:46 10.62.200.1 date=2026-05-10 time=15:59:46 devname="FortiGate-VM64-KVN" devid="FGVMEVRM27
T94VEB" eventtime=1778653986340731099 tz="-0700" logid="0100040704" type="event" subtype="system" level=
"notice" vd="root" logdesc="System performance statistics" action="perf-stats" cpu=0 mem=41 totalsession
=54 disk=0 bandwidth=0/0 setuprate=0 disklograte=0 faillograte=0 freediskstorage=0 sysuptime=186928 wan
info="name=port1,bytes=5805648/92237159,packets=48184/95218;" msg="Performance statistics: average CPU:
0, memory: 41, concurrent sessions: 54, setup-rate: 0"
May 11 07:04:46 10.62.200.1 date=2026-05-10 time=16:04:45 devname="FortiGate-VM64-KVN" devid="FGVMEVRM27
T94VEB" eventtime=1778654285264407118 tz="-0700" logid="0101037141" type="event" subtype="vpn" level="no
tice" vd="root" logdesc="IPsec tunnel statistics" msg="IPsec tunnel statistics" action="tunnel-stats" re
mip=172.16.23.2 locip=10.62.120.2 remport=4500 locport=4500 outintf="port1" cookies="08b9bda17bf72219/77
231b0721d745c0" user="N/A" group="N/A" useralt="N/A" xauthuser="N/A" xauthgroup="N/A" assignip=N/A vpntu
nnel="Site VPN" tunnelip=N/A tunnelid=4036935923 tunneltype="ipsec" duration=6316 sentbyte=252 rcvbyte=
252 nextstat=0
May 11 07:04:46 10.62.200.1 date=2026-05-10 time=16:04:47 devname="FortiGate-VM64-KVN" devid="FGVMEVRM27
T94VEB" eventtime=1778654286339390359 tz="-0700" logid="0100040704" type="event" subtype="system" level=
"notice" vd="root" logdesc="System performance statistics" action="perf-stats" cpu=1 mem=41 totalsession
=90 disk=0 bandwidth=1777 setuprate=0 disklograte=0 faillograte=0 freediskstorage=0 sysuptime=187228 wan
info="name=port1,bytes=6071513/94427363,packets=49228/97391;" msg="Performance statistics: average CPU:
1, memory: 41, concurrent sessions: 90, setup-rate: 0"
```

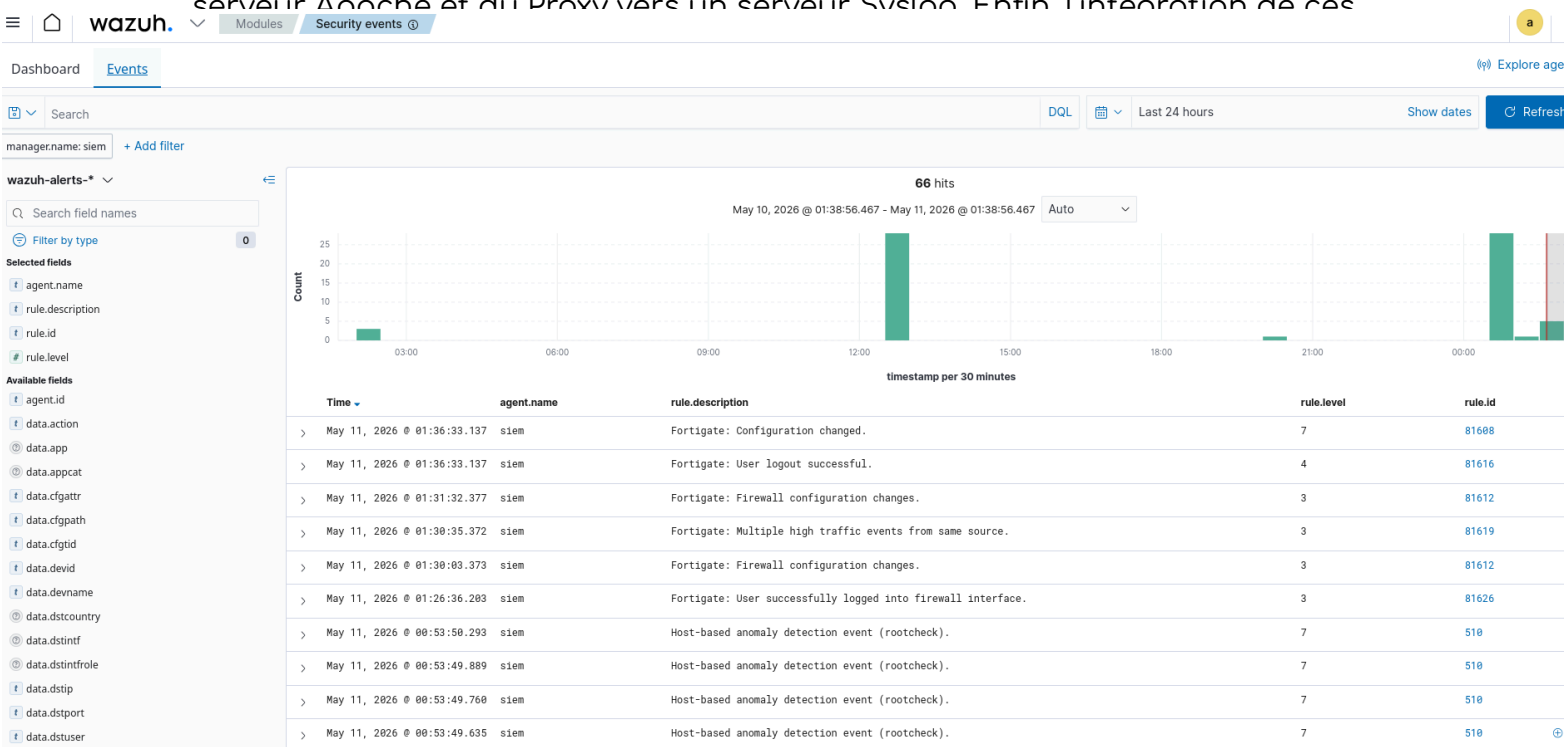
Résultat sur Wazuh:

Si un utilisateur tente de forcer l'accès à notre serveur Apache ou de contourner le Proxy, l'événement apparaît ici instantanément. Cela permet à l'administrateur de disposer d'une "Timeline" précise d'un incident de sécurité

sans avoir à se connecter individuellement sur l'interface d'administration de chaque équipement réseau.

Conclusion

Ce projet a permis de sécuriser une infrastructure réseau complète sur EVE-NG. On a d'abord protégé le serveur Web en configurant le HTTPS avec un certificat signé par l'Active Directory. On a ensuite mis en place une surveillance centralisée en redirigeant les logs du pare-feu Fortigate, du serveur Apache et du Proxy vers un serveur Syslog. Enfin, l'intégration de ces



Onglet 2

Sécurisation serveur web :

Installation de openssl : winget install openssl

Création du dossier certificats :

```
mkdir C:\certificats
```

```
cd C:\certificats
```

Création du certificat et de la clé :

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout apache-selfsigned.key -out  
apache-selfsigned.crt
```

```
C:\certificats>dir
Le volume dans le lecteur C s'appelle Windows-SSD
Le numéro de série du volume est 6CD4-4073

Répertoire de C:\certificats

28/04/2026  15:26    <DIR>          .
28/04/2026  15:26                1 270 apache-selfsigned.crt
28/04/2026  15:22                1 732 apache-selfsigned.key
                2 fichier(s)                3 002 octets
                1 Rép(s)  718 356 287 488 octets libres
```

La clé et le certificat sont bien créés.